

TT-GNN: Efficient On-Chip Graph Neural Network Training via Embedding Reformation and Hardware Optimization

Zheng Qu
UC Santa Barbara
USA
zhengqu.ece@gmail.com

Dimin Niu
Alibaba Group
USA
dimin.niu@alibaba-inc.com

Shuangchen Li
Alibaba Group
USA
shuangchen.li@alibaba-inc.com

Hongzhong Zheng
Alibaba Group
USA
hongzhong.zheng@alibaba-inc.com

Yuan Xie
Alibaba Group
USA
y.xie@alibaba-inc.com

ABSTRACT

Training Graph Neural Networks on large graphs is challenging due to the need to store graph data and move them along the memory hierarchy. In this work, we tackle this by effectively compressing graph embedding matrix such that the model training can be fully enabled with on-chip compute and memory resources. Specifically, we leverage the graph homophily property and consider using Tensor-train to represent the graph embedding. This allows nodes with similar neighborhoods to partially share the feature representation.

While applying Tensor-train reduces the size of the graph embedding, it imposes several challenges to hardware design. On one hand, utilizing low-rank representation requires the features to be decompressed before being sent to GNN models, which introduces extra computation overhead. On the other hand, the decompressed features might still exceed on-chip memory capacity even with the minibatch setting, causing inefficient off-chip memory access. Thus, we propose the TT-GNN hardware accelerator with a specialized dataflow tailored for on-chip Tensor-train GNN learning. Based on the on-chip memory capacity and training configuration, TT-GNN adaptively breaks down a minibatch into smaller microbatches that can be fitted on-chip. The microbatch composition and scheduling order are designed to maximize data reuse and reduce redundant computations both across and within microbatches. To mitigate TT computation overhead, we further propose a unified algorithm to jointly handle TT decompression during forward propagation and TT gradient derivation during backward propagation. Evaluated on a series of benchmarks, the proposed software-hardware solution is able to outperform existing CPU-GPU training systems on both training performance (1.55~4210 $\times$) and energy efficiency (2.83~2254 $\times$). We believe TT-GNN introduces a new perspective to address large-scale GNN training and enables possibilities to train GNN models even under a significantly constrained resource budget.



This work is licensed under a Creative Commons Attribution International 4.0 License.

MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0329-4/23/10.
<https://doi.org/10.1145/3613424.3614305>

CCS CONCEPTS

• **Computer systems organization** → **Neural networks**; • **Hardware** → **Application specific integrated circuits**; • **Computing methodologies** → **Symbolic and algebraic algorithms**.

KEYWORDS

Graph Neural Networks, Tensor-train Decomposition, Hardware Accelerator

ACM Reference Format:

Zheng Qu, Dimin Niu, Shuangchen Li, Hongzhong Zheng, and Yuan Xie. 2023. TT-GNN: Efficient On-Chip Graph Neural Network Training via Embedding Reformation and Hardware Optimization. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)*, October 28–November 01, 2023, Toronto, ON, Canada. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3613424.3614305>

1 INTRODUCTION

Originating from spectral graph analysis and fueled by the success of machine learning, graph neural networks (GNNs) have drawn a surge of interest and have been applied to various applications involving non-Euclidean graph-structured data. During the past few years, a wide range of GNN models [3, 10, 12, 38] have been proposed to solve graph-related problems. Exciting progress has been achieved by GNNs in domains such as recommendation systems [41], relation prediction [7], chemistry analysis [45], financial security [49], protein discovery [9, 36], EDA [16, 26, 27] and so on.

Despite the great application potential, training GNNs on large graphs is challenging due to the need to store graph data and move them along the memory hierarchy. Given the increasingly large problem size, minibatch training is currently the most widely adopted approach to train a GNN model[10]. As shown in Figure 1, each minibatch takes two steps. The first step is to sample a subgraph from the original graph. The structure of the subgraph and its corresponding node embeddings together form a minibatch of training data. In this paper, we consider the case where the graph is very large, such that the graph data are stored in a host system memory. Consequently, the subgraph preparation is handled by the host processor, such as a host CPU. After obtaining the minibatch training data, it is sent to training hardware such as GPU to execute the model training. In this second step, we perform forward and backward propagation on the subgraph to update model parameters.

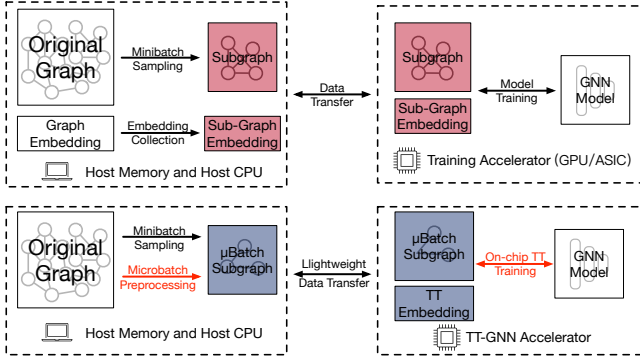


Figure 1: Illustration of typical minibatch training pipeline and TT-GNN training pipeline.

To speed up minibatch GNN training, prior works have proposed diverse software and hardware techniques targeting different stages of the training pipeline. Some work [4, 23, 48] aims at improving GNN computation efficiency with algorithmic and software optimizations. Others focus on reducing neighbor sampling latency [14] and data loading cost [1] to hide the subgraph preparation overhead. However, they all assume an unchangeable setting, that is each node of the graph should be independently represented by a feature vector. This assumption further leads to the explosion of the graph representation when the number of nodes scales to millions and billions. Eventually, memory capacity is saturated and training performance is compromised. According to our profiling experiments, collecting node features from the host memory can take 27.9 ~ 61.1% of the training time on a typical CPU-GPU system.

In this work, we tackle this problem by effectively compressing the graph feature matrix and storing it closer to computation resources for faster memory access. Specifically, we observe that different graph node features contain inter-relationships that can be well preserved even after applying low-rank approximation. Therefore, we consider using Tensor-train (TT) to represent the graph feature instead of using a 2D embedding matrix. In this way, we can represent the graph using a much more compact TT data structure while maximally preserving the representation capability. As shown in Figure 1, the resultant TT graph embedding can be stored in the accelerator’s on-chip buffer, and the embedding is jointly trained with the Graph Neural Network with much less memory consumption.

Although the algorithmic modification greatly reduces the memory cost of training GNNs, it imposes several new hardware challenges. (1) During the forward pass, TT-format embeddings need to be decompressed into the original vector format before being processed by the GNN model. Reversely, we also need to generate the TT-format gradient during the backward pass. Naively handling these TT-related computations is expensive, yet, exploring effective intermediate data reuse is non-trivial. (2) Although we can store the TT-format embedding in the on-chip buffer, the decompressed features used in each minibatch might still exceed on-chip memory capacity. Therefore, we need a more fine-grained dataflow to further split each minibatch into smaller compute graphs.

To tackle the aforementioned challenges, we propose TT-GNN, a training system that incorporates software and hardware co-optimizations for efficient GNN learning at scale. Firstly, to mitigate TT computation overhead, we propose a unified algorithm to jointly handle TT decomposition and TT gradient derivation. The proposed algorithm can be flexibly configured to be more compute-efficient by caching more reusable results, or more memory-efficient by tolerating some recomputation overhead. Secondly, by evaluating on-chip memory capacity and training configuration, TT-GNN dynamically breaks down a minibatch into smaller microbatches that can be fitted on-chip. To reduce redundant computations caused by neighbor sharing across different microbatches, we cache the last few layers of the GNN model on-chip, and only fan out from an intermediate layer if necessary. The microbatch composition and scheduling order is designed to maximize data reuse both across and within microbatches. Finally, we explore the reuse opportunities of aggregated partial sums which benefit both neighbor aggregation in forward propagation and gradient scattering in backward propagation.

Combining the algorithm and architecture co-design, TT-GNN achieves 1.55~4210× training speedup and 2.83~2254× energy efficiency improvements compared with the baseline CPU-GPU system on a series of GNN benchmarks. The key contribution of this work is summarized as follows:

- We perform in-depth characterization of GNN training on a standard CPU-GPU system, locating the training pipeline bottleneck being the feature collection and uncovering the underlying causes.
- Based on profiling results, we propose to compress the feature matrix such that it can be held in faster memory. We also conduct preliminary experiments to demonstrate the benefit of performing on-chip decompression over retrieving the feature from off-chip memory.
- We propose a training system with software hardware co-optimizations tailored for efficient GNN training. In our design, only the graph sampling is executed in the host system, while the graph embedding collection, as well as GNN training, are fully handled on-chip.
- We evaluate TT-GNN on multiple GNN datasets, demonstrating the effectiveness of the proposed design and the possibility to train large GNNs with limited resources.

2 BACKGROUND AND MOTIVATION

In this section, we first present the basics of Graph Neural Networks. We then introduce our in-depth GNN training characterization on a GPU system, which motivates us to propose TT-GNN.

2.1 GNN Basis and Minibatch Training

We first introduce the basic of GNNs. Given an undirected graph, we denote it as $G = (V, E)$, where $|V|$ is the number of nodes and $|E|$ is the number of edges in the graph. Each node is described by a feature vector of length F , and all the node features together forms a 2D feature matrix $X \in \mathbb{R}^{|V| \times F}$. In most cases, matrix X is dense and of large-scale due to the massive amount of nodes contained in real-world graphs.

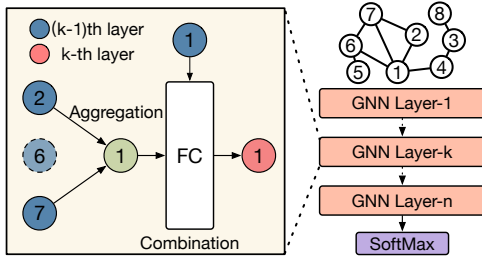


Figure 2: Illustration of a sample GNN model.

During GNN processing, each GNN layer follows a two-stage procedure, namely *Aggregation* and *Combination*. As shown in Figure 2 and equations in below, each node v will collect feature vectors from its sampled neighborhood $N(v)$ to generate an aggregated feature a_v^k . The aggregation operator can be flexibly designed, where common choices include *Mean*, *Max*, *MLP* and so on. After this, the aggregated feature is combined with source node v 's feature vector $h^{(k-1)}_v$. The combination operator utilizes these two vectors to generate hidden representation $h^k(v)$ of node v .

$$\begin{aligned} a_v^k &= \text{Aggregate}(u : u \in N(v) \cup v) \\ h_v^k &= \text{Combine}(a_v^k, h_v^{(k-1)}) \end{aligned}$$

To train a GNN model, we typically adopt the minibatch strategy. As illustrated in Figure 3, for each minibatch, we fan out from a group of target nodes. When considering the receptive field, we sample a fixed-size set of neighbors instead of using the full neighborhood for each node. This results in a funnel-shaped network, where the cost of each layer follows a decreasing order. To perform the GNN computation, we start from the input nodes of the first layer, use their feature vectors and follow the graph structure to perform aggregation and combination. The generated hidden node features will be further used as the input to the next layer.

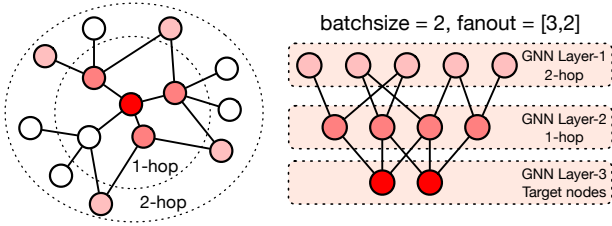


Figure 3: Sampling-based minibatch GNN training.

2.2 GNN Training Characterization

As mentioned above, there are mainly two types of data used in minibatch GNN training, the graph structure represented in CSR format, and corresponding feature embedding stored in a 2D matrix. Since a real-world graph may contain a massive amount of nodes and edges, both graphs CSR and embedding matrix can consume large memory space.

We observe that the location of the graph data significantly affects the overall training performance. When both graph structure and embedding matrix can be fit into GPU device memory, we can directly perform sampling and feature collection on GPU [39], therefore avoiding transferring data between host memory and device memory. However, if the data exceeds GPU's memory capacity,

the sampled data will have to be sent via the system interconnect (e.g., PCIe). To illustrate the performance gap, we conduct a profiling experiment using a popular GNN model (GraphSAGE [10]) and a real-world benchmark (ogbn-products [11]). The model is implemented in DGL [39], and experiments are done on an Nvidia 3090 GPU using Nsight System.

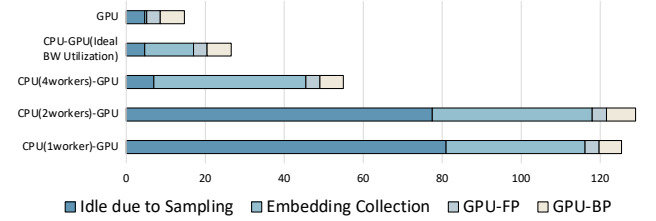


Figure 4: Average Latency(ms) Breakdown of Training One Minibatch on 3090. The batchsize is set to 500, with a 3-hop neighbor fan-out of [5, 10, 15]

Figure 4 shows the training latency comparison when the graph is stored in GPU HBM or in the host DRAM. The end-to-end latency is broken down into different steps. As we can see from the figure, under the same batchsize, for each epoch, training on HBM is $3.74 \sim 8.77\times$ faster than training on host DRAM. The performance difference purely comes from the sub-graph preparation stage. When the graph is completely stored in HBM, GPU performs parallel graph sampling and directly fetches node features from HBM. Therefore, the combined latency of sampling and feature collection is shorter than the latency of forward and backward propagation. This further indicates opportunities to fully hide the subgraph preparation overhead with pipelined execution.

On the contrary, CPU-based graph sampling and feature collection are much slower, uncovering the subgraph preparation cost. To improve graph sampling efficiency, we can issue multiple threads (#worker) to simultaneously perform sampling for different minibatches. The generated subgraphs are stored in a task queue to be fetched later. As a result, when #worker is set to 4, the per-minibatch sampling latency only consumes 15.5% of the total training time, as opposed to 61.8% in single thread implementation. However, compared with graph sampling, it is non-trivial to address the embedding collection overhead. The datapath is inevitably longer as we need to first copy the features from host memory to device memory through PCIe. This additional step is long enough to be a deal breaker of a perfect execution pipeline.

In our experiments, we also notice that the feature collection kernel does not fully saturate PCIe bandwidth due to insufficient memory requests to be issued. As shown in the Table below, the average PCIe bandwidth utilization for different batchsizes is $32.1 \sim 35.2\%$. Therefore, we projected a theoretical lower bound of feature collection latency as shown in the second line of Figure 4. The result indicates that improving PCIe utilization with locality-enhancing techniques such as graph partitioning is beneficial, but insufficient to address the problem, as the total latency of sub-graph preparation is still longer than the combined latency of GPU forward and backward propagation.

In summary, to fully address the subgraph preparation problem, a more effective way is to shorten the datapath by storing the embedding matrix closer to computation resources. In this work, we

Table 1: Avg. PCIe utilization under different batchsizes.

BATCHSIZE	500	1000	2000	4000
UTILIZATION(%)	33.18	32.10	34.10	35.20

achieve this by utilizing a much more compact embedding representation structure. We also customize the system dataflow and hardware accelerator which enables a more efficient on-chip GNN training scheme.

2.3 TT Decomposition and TT Representation

Before going into the details of TT-GNN, we introduce the fundamental idea of using Tensor-train Decomposition (TTD) to compress a matrix. TTD has been originally proposed as a generalization of Singular Value Decomposition for high order tensors [32]. Given a d -dimension tensor $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_d}$, TTD decomposes it into a sequence of 3-dimension tensors. Therefore, each scalar in $\mathcal{A}$ can be derived as follows:

$$\mathcal{A}(i_1, i_2, \dots, i_d) \approx \mathcal{G}_1(:, i_1, :) \mathcal{G}_2(:, i_2, :) \dots \mathcal{G}_d(:, i_d, :). \quad (1)$$

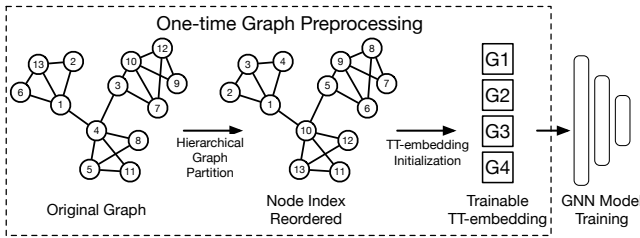
$\mathcal{G}_k$ is a tensor of size $r_{k-1} \times I_k \times r_k$, where r_k is called the TT-rank. r_0 and r_d are set to 1 such that the product of the above matrix sequence is a scalar. Other TT-ranks can be either predefined before the decomposition or decided during runtime according to the required decomposition accuracy. Higher TT-ranks increase the decomposition accuracy but also increase the size of the TT-format representation.

Apart from decomposing tensors, TTD can also be utilized to deal with large vectors and matrices. Specifically, in order to apply TTD on a matrix X of size $M \times N$, we need to factorize M into $\prod_{k=1}^d m_k$ and factorize N into $\prod_{k=1}^d n_k$. This allows us to reformat matrix X as a $2d$ -dimension tensor $\mathcal{X} \in \mathbb{R}^{(m_1 \times m_2 \times \dots \times m_d) \times (n_1 \times n_2 \times \dots \times n_d)}$. Thus, the matrix can now be decomposed with TTD and represented as follows:

$$\mathcal{X}((i_1, j_1), (i_2, j_2), \dots, (i_d, j_d)) \approx \mathcal{G}_1(:, i_1, j_1, :) \dots \mathcal{G}_d(:, i_d, j_d, :). \quad (2)$$

Prior works have leverage TTD to compress weight matrices in Neural Network models, such that the number of model parameters is significantly reduced [8, 29, 31, 46].

3 TT-FORMAT GNN TRAINING

**Figure 5: Illustration of TT-GNN workflow.**

In this section, we introduce the workflow of applying Tensor-train decomposition on Graph Neural Networks, which is originally

proposed in [47]. Essentially, we need to add a one-time preprocessing step prior to the model training to define a trainable TT-format embedding. The key idea is to align graph topological information with the Tensor-train data structure. Specifically, as shown in Figure 5, we first perform a hierarchical graph partition (e.g., METIS [15]) to group the nodes into multiple levels of clusters. Then, we reorder the graph nodes based on the partition results, such that nodes in the same partition will have continuous indices. In this way, we can directly reflect graph homophily in the embedding representation. For example, suppose we apply a three-level METIS partition over the graph, which results in a $[10, 10, 10]$ index system. In this setting, node 101 will be mapped to $[1, 0, 1]$, and its embedding will be represented by $\mathcal{G}_1(:, 1, :, :)\mathcal{G}_2(:, 0, :, :)\mathcal{G}_3(:, 1, :, :)$. Similarly, node 102 will be mapped to $[1, 0, 2]$, and node 312 will be mapped to $[3, 1, 2]$. As a result, node 101 and 102 will share the first two tensor core representations, while being more different from node 312. In this way, we are able to adjust the degree of feature sharing across different nodes by reordering the node indices according to the neighborhood similarity.

Originally, each node is represented with a feature vector of length F , and all the node features together form a 2D feature matrix $X \in \mathbb{R}^{N \times F}$ ($N = |V|$). By applying TTD to X , the feature matrix is now represented as:

$$X = \mathcal{G}_1 * \mathcal{G}_2 * \dots * \mathcal{G}_d \quad (3)$$

where $\mathcal{G}_i \in \mathbb{R}^{r_{i-1} \times n_i \times f_i \times r_i}$, $N = \prod_{i=1}^d n_i$ and $F = \prod_{i=1}^d f_i$.

To extract the k^{th} row from the feature matrix, it is equivalent to first finding the projection index $(n_0^k, n_1^k, \dots, n_d^k)$, fixing each corresponding n -index in $\mathcal{G}_k$, and finally calculating the product of the tensor sequence.

$$X(k, :) = \mathcal{G}_1(:, n_0^k, :, :)\mathcal{G}_2(:, n_1^k, :, :)\dots\mathcal{G}_d(:, n_d^k, :, :)\quad (4)$$

Finally, as shown in Figure 5, after defining the TT embedding structure and node indices with graph partitioning results, we further need to initialize the TT-format embedding parameters. Prior work [47] has demonstrated the superiority of orthogonal initialization regarding convergence effectiveness. In TT-GNN we adopt the same strategy to initialize the parameters, and the TT-format embedding will be jointly trained with the GNN model.

3.1 Compression Ratio and Model Accuracy

Since Tensor-train allows partial feature sharing across graph nodes, it is naturally a much more compact embedding representation. Before we need $O(NF)$ space to store the uncompressed features, with TT-GNN, we only need $O(dNf_i r^2)$ elements to represent all the node features in the graph. To provide an intuition, Reddit[10] contains 232965 nodes and the length of each feature vector is 602. In our experiments, we have $d = 7$, $r = 5$, n_i and f_i within $[3, 5]$. Therefore, the compression ratio is 60976 $\times$, reducing the size of the embedding matrix from 534.99 MB to 8.98 KB.

In the table below we list the accuracy and compression ratio (CR) of TT-GNN on different benchmarks. We compare TT-GNN with two baselines, **ORIG EMB** means training the GNN model on the original embedding matrix, and **TRAINABLE** means training a 2D embedding together with the GNN model. As we can see from the results, TT-GNN achieves orders of magnitude compression

ratio and better accuracy compared with 2D trainable embeddings. On the other hand, applying TT causes accuracy degradation on certain benchmarks. Overall, TT-GNN is more suitable under the scenario where we lack node features, thereby requiring learning the embeddings during training [47].

Table 2: TT-GNN Accuracy and Compression Ratio.

DATASET	ORIG EMB	TRAINABLE	TT	CR
CORA	81.4%	60.7%	78.1%	6189×
REDDIT	95.6%	91.1%	93.3%	60976×
OGBN-ARXIV	72.3%	72.1%	72.2%	32546×
OGBN-PRODUCTS	78.9%	73.4%	74.2%	132268×

4 CHALLENGE AND OPPORTUNITY

In this section, we describe the opportunities and challenges when adopting TT-GNN for efficient training of Graph Neural Network models. We also present the experiments and preliminary analysis that we conducted, which leads to the dedicated architecture and dataflow in the following section.

The straightforward benefit of using a compressed format embedding is that we can store it closer to the compute unit, thus reducing the time required for fetching these embeddings for training. As mentioned earlier in section 2, moving the embedding to GPU’s HBM is efficient enough to hide the embedding fetching latency. While this seems to be a free lunch for TT-GNN, it also leads to new hardware challenges.

Decompression Overhead: The new TT-format embedding brings us a significant compression ratio but also introduces computation overhead when we decompress the TT-feature back to the original feature vector. As shown by equation 4, fetching one feature vector now becomes a sequence of matrix multiplication, as we need to gradually contract out all the rank dimensions when recovering the embedding. To provide some intuition over the cost, we compare the theoretical decompression complexity to the computation cost of forward propagation of the GraphSAGE [10] model on the Reddit dataset. The GraphSAGE model has two graph convolution layers, with a neighbor fan-out to be $\{10, 25\}$. The forward function can be expressed as equation 5. Since TT-rank affects the computation complexity of the decompression, we sweep over multiple possible rank values. We also select different batchsizes as it will influence the portion of shared neighbors, and eventually the decompression complexity as well.

$$h_v^k = \sigma(\mathbf{W} \cdot \text{MEAN}(\{h_v^{k-1}\} \cup \{h_u^{k-1}, \forall u \in \mathcal{N}(v)\})) \quad (5)$$

The results are shown in Figure 6. For each minibatch size and each rank value, we normalize the computation cost of TT-decompression to the cost of forward propagation. The first thing to be noticed is that, TT computation overhead increases exponentially with the rank values. The cost of decompressing one minibatch is almost the same as running the whole network when TT-rank is equal to 10, not to mention an even larger rank value. Secondly, TT-GNN is in favor of larger minibatch sizes. This is because when more target nodes are considered in one minibatch, they will share more common neighbors at the input layer, resulting in sublinear

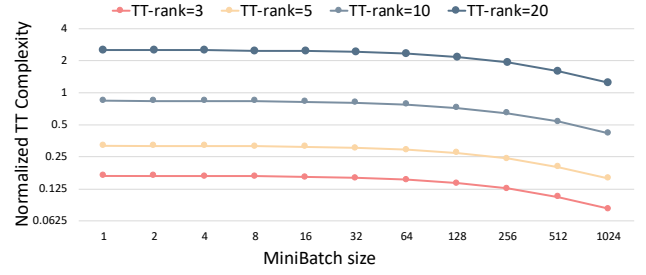


Figure 6: Per-minibatch computation complexity of TT decompression relative to the forward propagation complexity of a two-layer GraphSAGE model.

Table 3: Energy consumption comparison between fetching original feature from off-chip HBM and decompressing corresponding TT-feature from on-chip SRAM.

DATASET	TT(R=3)	TT(R=5)	TT(R=10)	TT(R=20)
SRAM(pJ)	633	1339	4099	13882
TT-DECOMP.(pJ)	13082	41860	222640	1332160
HBM(pJ)	64075	64075	64075	64075

increase of the input nodes. On the other hand, the forward propagation cost is mainly affected by the number of sampled edges, which is decided by the preset fan-out as long as the nodes have enough neighbors to be sampled. In conclusion, the decompression of TT-GNN can have a comparable cost as running the GNN model, thus should be efficiently handled.

Trading Computation for Memory Efficiency In the problem above we argue that TT-GNN prefers a larger minibatch size, as more shared neighbors help avoid redundantly decompressing the same input nodes. However, as we further show with Table 3, this strategy only holds true when prior decompressed features can be cached on-chip. In Table 3 we compare the energy consumption of accessing one original feature vector from HBM, with the energy consumption of accessing the corresponding TT-format embedding in an SRAM buffer and decompressing it on-chip. For HBM estimation, we borrow the data from prior work [33] and assume a 3.97 pJ/bit of energy consumption. We use CACTI [28] to get the simulated result of the SRAM buffer and borrow data from prior work [5] to estimate the energy consumption of floating point operations. From the comparison, we find that when using a relatively small rank value, directly performing TT-decompression on-chip consumes less energy compared with fetching the feature vector from off-chip memory. This indicates a potential design choice to eliminate off-chip feature access by performing TT decompression whenever needed. The challenges, however, are of two folds. On one hand, using small rank values will introduce larger compression errors and cause a negative impact on model accuracy. On the other hand, replacing memory access with TT decompression will cause a massive amount of features to be recomputed. We want to reduce such repetitive computation as much as possible by efficiently utilizing limited on-chip memory.

5 TT-GNN TRAINING DATAFLOW

To exploit the algorithmic potential of TT, we present the TT-GNN dataflow in this section. Overall, we address the training problem with a top-down design, as we gradually decompose the problem

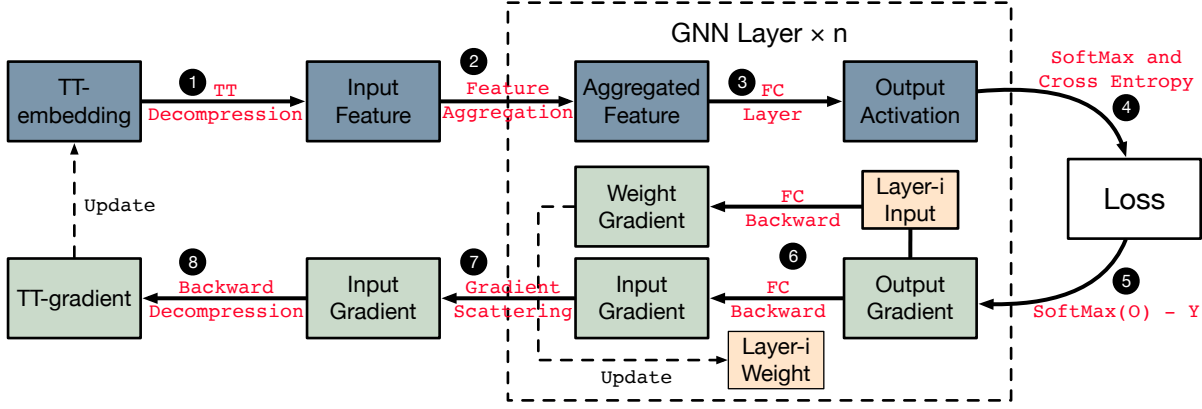


Figure 7: TT-GNN Training dataflow

to be fitted on-chip. Specifically, the proposed dataflow mainly consists of three main parts. (1) To completely eliminate off-chip memory access under dynamic training configurations (e.g., minibatch size, GNN configuration), we introduce the Hybrid Minibatch-Microbatch tiling strategy to adaptively control the size of the sub-graph being trained on the accelerator. To reduce the redundant computations caused by neighbor sharing across minibatches, as well as maximize data reuse within each microbatch, we customize the microbatch composition and scheduling order. (2) We propose a unified algorithm to handle TT decomposition during forward pass and TT-gradient computation during backward pass. The proposed algorithm exploits data reuse among these two operators and provides a flexible mechanism to trade-off between compute efficiency and memory consumption. (3) Finally, we improve the aggregation and gradient scatter efficiency by offline reorganizing the microbatch subgraph as soon as it is generated. In this section, we provide a detailed walkthrough of our TT-GNN dataflow assuming a two-layer (two levels of neighbor fan-out) GraphSAGE model with a *MEAN* function as the aggregating operator.

5.1 Highlevel Training Dataflow

Figure 7 presents the computation graph of a TT-GraphSAGE model. We use squares to indicate the data at each layer and use arrows to illustrate the operations that transform these data between each other. As shown in the figure, ❶ the forward propagation starts with a TT-layer, where the TT-format embeddings will be decompressed into a minibatch of input vectors to be sent to the model. The decompression operation, as we show in section 2, is essentially a sequence of small tensor contractions which can be implemented as matrix multiplications. ❷ After we obtain these input feature vectors, each node in the hidden layer will fetch its neighbor features and perform the aggregation function. In this case, the aggregation is simply a *MEAN* function. ❸ The aggregation is followed by an Apply function, where typically the hidden node feature and the aggregated neighbor feature are combined together using a Fully Connected layer to generate the hidden node features. This two-step message passing is repeated n times depending on the number of hidden layers in the GNN model. ❹ Finally, we apply the SoftMAX operation to obtain the final classification result.

❺ Reversely, the backward propagation starts from the classification loss and ends at the TT-layer. ❻ At each GNN layer, the output gradient is first propagated through the NN layer with matrix multiplication. ❼ Then, the hidden feature gradient needs to be scattered back to the input nodes. In other words, the gradient of each hidden node will be scattered and accumulated to all the used input nodes during the forward aggregation. ❽ Finally, after the gradient of the model input features is obtained, we use equation 6 to compute the gradient of TT-embeddings.

$$\frac{\partial L}{\partial G_i(:, n_i^k, :, :)} = \prod_{j=1}^{i-1} G_j(:, n_j^k, :, :) * \frac{\partial L}{\partial X(k, :)} * \prod_{j=i+1}^d G_j(:, n_j^k, :, :) \quad (6)$$

5.2 From Minibatch to Microbatch

As illustrated in Figure 8 (a), the biggest difference between minibatch GNN training and conventional full-batch GCN training is the inconsistent cost of each layer caused by neighbor fan-out. Due to the neighbor sampling mechanism, there will be more and more nodes and edges as we approach the input layer. This also indicates an increasing memory and computation cost. The selection of minibatch size, which is essentially the number of destination nodes (2 in this sample), will also affect the sampled graph size and the corresponding minibatch training cost. Generally, as shown by Figure 8 (b), when we process the who minibatch layer by layer, if any of the layers exceeds on-chip memory capacity, we will have to use off-chip memory for temporary storage. The white circles indicate node features stored off-chip, and red dashed lines represent associated off-chip memory access. with Tensor-train format embedding and on-chip decompression, we naturally eliminate inefficient off-chip embedding loading, as shown in Figure 8 (c). However, the intermediate node features can still cause off-chip storage. Therefore, we propose to further break the minibatch into smaller groups which we called **microbatch**, which can be completely fitted on-chip.

Intuitively thinking, a microbatch can be obtained by simply selecting a portion of the destination nodes from the original minibatch. As shown in Figure 8 (d), a smaller subgraph can be sampled from the selected nodes and their neighborhoods. This is equivalent to setting the minibatch size to be a smaller value in the first

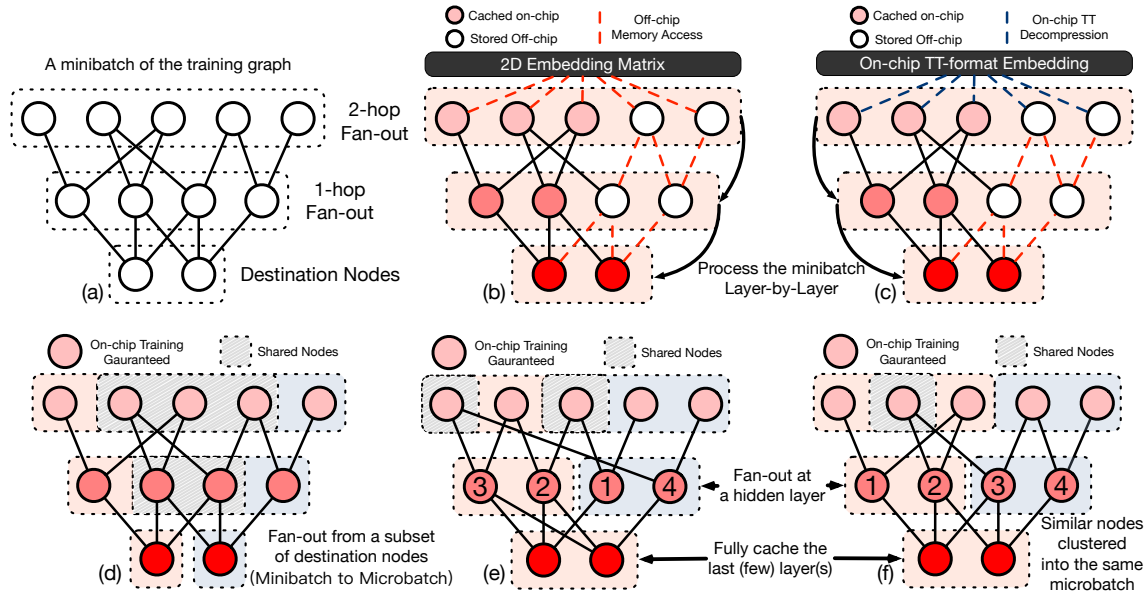


Figure 8: Step-by-step walk-through of TT-GNN's Hybrid Minibatch-Microbatch on-chip training dataflow.

place, except that we do not update the model parameter after the backward pass of the microbatch. However, this naive strategy will incur redundant computations and memory access across different microbatches. In this example, suppose we are breaking this minibatch with 2 destination nodes into two microbatches, each with 1 destination node. Due to neighborhood sharing, although the destination nodes of the two microbatches are completely different, they could share common nodes in the hidden layer, and even more in the input layer. Consequently, all the computations related to these shared nodes will be redundantly computed unless we can cache the previously computed node features. However, the limited on-chip memory capacity only provides us with a tight reuse distance budget. Even if we can cache the shared nodes, the memory access over the shared nodes is still inevitably repeated across different microbatches. The situation gets worse with larger batchsize, deeper network architecture, and with the added TT-layer at the beginning.

To tackle the above-mentioned challenge and enable efficient on-chip training with as little overhead as possible, we propose our Hybrid Minibatch-Microbatch tiling strategy.

Hybrid Minibatch-Microbatch Tiling: As presented in Figure 8 (e), the first insight is that the last few layers in a GNN model are much smaller compared with the beginning layers. Thus, the cost of caching all the destination nodes and their close neighbors is relatively low. Therefore, instead of breaking the minibatch directly from the output layer, we keep the last few layers the same as the original minibatch and start tiling at an intermediate layer. In this example, we reserve the space for all two destination nodes and break the minibatch into microbatches at the hidden layer. The benefit is obvious. As shown in figure (e), for each microbatch, after the target hidden nodes are generated, it can be directly used to compute the last layer. The hidden node feature will be added to

the partial sum of the destination nodes which are always on-chip. In this way, there will be no shared hidden nodes across the microbatches, and all the hidden node features only need to be computed and used one single time. We call this Hybrid Minibatch-Microbatch Tiling as it works in a microbatch fashion at first but eventually merges into the minibatch output. Another benefit of using this strategy is that it reduces the number of shared neighbors at the first (few) layers. As shown by the example in Figure 8 (e), since each microbatch contains less nodes compared with (d), the shared neighbors in the input layer are also reduced, which leads to fewer redundant TT-decompression.

The method works similarly in backward propagation. First, the gradient of the hidden nodes only needs to be computed and stored for one time as there is no neighbor sharing across microbatches. On the other hand, for shared neighbors in the first layer, the gradient derived from one microbatch is only a partial sum. We seek to avoid caching these partial sums to be accumulated because the first layer is the most memory-consuming layer. Therefore, we can directly use the gradient in each microbatch to derive the TT-format gradient of the TT embeddings. The TT-format gradient consumes much less space and is always stored on-chip. An exception is that we will delay the computation of TT gradient only if we know the gradient of a specific node will be accumulated in the next consecutive microbatch (only consider one-step reuse). This information is available to us as we decide the composition and scheduling order of the microbatches when we perform minibatch sampling. In either way, we avoid caching the vector format gradient of the first layer, so that to control memory consumption.

Microbatch Selection and Scheduling Order As mentioned above, the shared neighbors in the first few layers can still cause redundant TT decompression and TT-gradient computation. To address the problem, we further propose to customize the microbatch

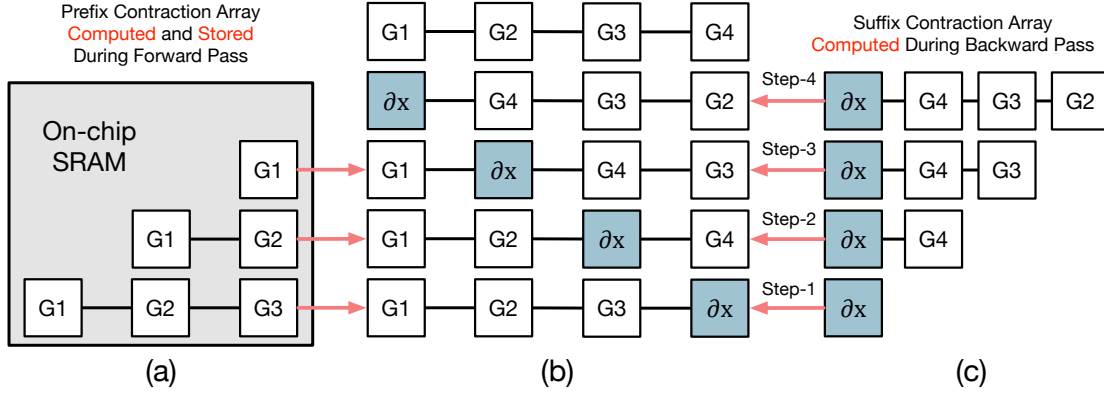


Figure 9: The contraction flow of TT decomposition as well as the gradient computation during the backward pass.

composition and scheduling order to maximize intra- and inter-microbatch data reuse. Figure 8 (d) and (e) provide an illustration. Originally in Figure 8 (d), we group node 3 and node 2 into one microbatch, and group node 1 and 4 in another microbatch. This results in two shared neighbors at the first layer. One solution is to schedule these two microbatches next to each other, so that the shared neighbors can be cached on-chip and reused. For another solution, as shown in Figure 8 (f), we can group nodes with similar neighborhoods into the same microbatch. In this case, if we select node 1 and 2 to be the first microbatch, and node 3 and 4 to be the second, then there would be only one shared neighbor across these two microbatches. Reducing the overhead of redundant computation even if these two microbatches are not processed consecutively.

As we can see, these two strategies tackle the problem at different levels. Thus, in TT-GNN, we combine them into a unified strategy. Recall that at the beginning of the TT-GNN training, we first reorder the graph nodes according to the METIS partition results. Therefore, the reordered node index naturally indicates neighborhood similarity. In other words, nodes with close index values should be grouped into the same minibatch. Therefore, given a set of hidden nodes to be scheduled, we first sort these nodes according to their indices. After this, we can simply traverse the index list and group consecutive nodes into one microbatch. Besides, the consecutive microbatches will also be scheduled sequentially. In this way, we can efficiently obtain the microbatch composition as well as scheduling order together with one single pass.

5.3 Microbatch Dataflow Walk-through

In the above subsections, we break the minibatch into microbatches with minimized overhead, such that the microbatch can be completely processed on-chip. We further argue that there still exists performance improvement opportunities within each microbatch. Therefore in this subsection, we walk through the forward and backward pass of each microbatch to illustrate our intra-minibatch optimizations.

TT Decompression and Update As shown in Figure 7, TT decomposition is required during forward propagation, and during the backward pass we need to compute the TT-gradient to update

TT-embeddings. The corresponding equations of the two operations are presented earlier in equation 4 and 6.

We observe that both TT decomposition and TT-gradient can be considered as contracting a tensor-train network. We use Figure 9 as an illustration. In this example, we have four TT cores. To obtain an input feature vector from the TT-embedding, we need to extract a small tensor from each TT-core that together forms a tensor-train network. This is shown as the top tensor-train ($G1 - G2 - G3 - G4$) in Figure 9 (b). On the other hand, in the backward pass, we need to separately compute the gradient of the four tensors, which is represented as the bottom four tensor-trains in Figure 9 (b). As we can see, although the operation is still tensor-train contraction, one of the tensors should be replaced by the gradient of the feature vector.

To effectively explore data reuse in this problem, we propose to compute the required tensor-trains with the combination of prefix and suffix array. As shown in Figure 9 (a), during forward pass, we use an array to store the intermediate prefix contraction results. On the contrary, we only need to maintain a single suffix contraction result to generate the output gradient of each tensor. For example, as shown in Step-1 of Figure 9 (c), we first use the vector gradient and the cached $G1 - G2 - G3$ to generate the last tensor-train. Then, we update the suffix contraction result by multiplying ∂x with $G4$, and use another cached prefix result to generate the next tensor-train. Eventually, we can obtain all the TT-gradients with the stored prefix array and a suffix contraction result.

Note that, we are able to flexibly trade-off between compute efficiency and memory consumption with this algorithm. For example, we can choose to skip storing the prefix array during the forward pass and recompute it in the backward pass. This can significantly reduce the memory cost. On the other hand, we can simultaneously compute the prefix and suffix array in the forward pass, thereby reducing the sequential computation flow in the backward pass at the cost of higher memory consumption.

Neighbor Aggregation and Gradient Scatter Neighbor aggregation and gradient scatter are two important operations in a GNN model. During forward pass, we collect the neighbor information of each target node and generate the aggregated feature vector. In the back pass, we need to scatter the gradient of the target node back to all its neighbors. From a message flow perspective, these two

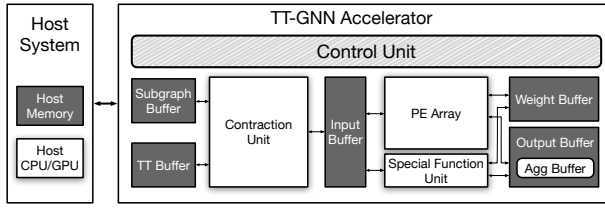


Figure 10: Overview of the TT-GNN Training System.

operations are reversed from each other. However, computationally both of them can be formulated into a Sparse-Dense Matrix Multiplication (SpMM) operation, where the sparse matrix operator is the adjacency matrix of the subgraph. Moreover, the sparse matrix of the scatter SpMM is simply the transpose of the aggregation SpMM.

To improve the compute efficiency of such SpMM operation, prior works have proposed searching algorithms [2, 13] to exploit intermediate data reuse. The key idea is to introduce a new set of aggregation nodes, where these nodes are essentially the partial sums of the input nodes. By identifying the popular partial sums as the aggregation nodes, we can avoid redundantly aggregating the associated input features, with very little memory overhead. In TT-GNN, we use a similar method but apply it to both forward pass and backward pass to save computations.

6 SYSTEM AND ACCELERATOR ARCHITECTURE

In this section, we introduce the complete design of the TT-GNN training system. The overall system-level architecture is presented in Figure 10. The proposed training dataflow is implemented in a dedicated accelerator, which is further attached to a host processor. Since TT-GNN does not compress the graph structure, the adjacency list is stored in the host memory. During training, the host processor is responsible for sampling minibatches from the graph adjacency list. To facilitate an efficient on-chip learning procedure, the host processor will further execute two tasks. (1) It will analyze the memory consumption of the minibatch and decompose the minibatch into microbatches if necessary. The procedure used for microbatch selection and scheduling is already discussed above in Section 5.2. (2) After the microbatches are decided, the host processor will further preprocess the compute graph to identify the intermediate aggregation set. As we mentioned in Section 5.3, this helps improve the SpMM efficiency. As soon as one microbatch is generated, it will be pushed to a task queue together with the dataflow configuration. The accelerator will execute the microbatch training based on the scheduled tasks. At the same time, the host processor can simultaneously prepare multiple minibatches and generate the associated microbatches.

As shown in Figure 10, TT-GNN accelerator mainly consists of the following modules: (1) A Contraction Unit that handles TT-decompression and TT-gradient computation. (2) A PE Array that is responsible for GNN related operations, including FC forward and backward computation, neighbor aggregations, as well as gradient scattering. (3) On-chip SRAM modules that store different types of data, including TT embeddings, microbatch subgraph structure, dataflow configuration, node features, model parameters, and all

Table 4: Summary of dataset statistics

DATASET	#NODE	#EDGE	#LABEL	FEAT LEN
CORA	2,708	10,556	7	1,433
REDDIT	232,965	114,615,892	41	602
OGBN-ARXIV	169,343	1,166,243	40	128
OGBN-PRODUCTS	2,449,029	61,859,140	47	100
OGBN-PAPERS100M	111M	1,615M	172	128

the computed gradients. (4) An overall Control Unit that orchestrates the memory and computation resources using the dataflow configuration file provided by the host processor.

Contraction Unit and PE Array Although the TT Contraction Unit and the PE Array handle different stages of the GNN training, the underlying computation pattern is common. For TT-decompression and TT-gradient computation, the operation is tensor-train contraction, which can be further decomposed into sequences of matrix multiplications. For GNN-related computation, the PE array takes care of matrix multiplication in the FC layer and the vector-wise addition used during aggregation and gradient scattering. Therefore, both TT contraction Unit and PE Array adopt a classic 2D Mac array architecture so that we can efficiently map the parallel vector operations to the modules. We decouple the design of the Contraction Unit as well as the PE array so that they can operate in a pipelined manner. Since we do not need to update the TT-embeddings across different microbatches, we can decompress the input node features for the next microbatch while processing the forward and backward pass of the current microbatch.

Special Function Unit The Special Function Unit incorporates floating point arithmetics to handle functions including division, exponential operations, modular operations, and so on. These operators are composed together to implement SoftMax function, index projection between node IDs and TT-index, Optimizer-related computations (e.g., parameter update in Adam [18]), batch normalization, and so on.

On-chip Memory TT-GNN has multiple on-chip SRAM buffers for storing different types of data used during training. The TT-embeddings and TT-gradients are stored in *TT-Buffer*. The microbatch graph structure, as well as the dataflow configuration file generated by the host processor, are stored in the *Subgraph-Buffer*. The *Input-Buffer* caches the decompressed input node features before being processed by the GNN model. It also stores the vector-format feature gradient. *Weight-Buffer* stores GNN model parameters and parameters gradients. *Output-Buffer* caches all the activation maps as well as the gradients of the hidden nodes. Finally, we specifically allocate a fraction from the *Output-Buffer* as the *Aggregation-Buffer* to store intermediate aggregated partial sums. As we discussed in Section 5.3, this improves the computation efficiency of Neighbor Aggregation and Gradient Scattering. The size of the *Aggregation-Buffer* is configurable depending on the benchmark characteristics. This information is obtained from microbatch generation and is included in the microbatch configuration file.

7 EVALUATION METHODOLOGY

In this section, we present the designed experimental methodology to evaluate TT-GNN.

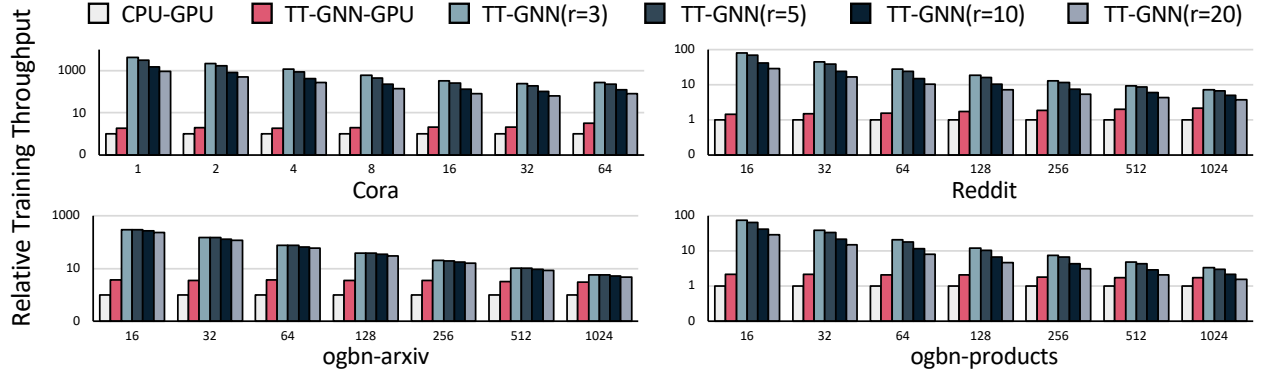


Figure 11: Relative training throughput compared with baseline CPU-GPU system and TT-GNN GPU implementation

7.1 Benchmark and Implementation

The baseline CPU-GPU training pipeline and TT-GNN GPU solution are implemented with Deep Graph Library [39]. We also implement the proposed Microbatch generation and preprocessing strategy in software and integrate it into the training pipeline. We use GraphSAGE [10] as the model architecture and select a series of GNN benchmarks to evaluate TT-GNN, including Cora, Reddit, and three node property prediction datasets from Open Graph Benchmark [11]. The basic attributes of each graph benchmark are listed in Table 4.

7.2 Hardware Performance

Hardware Implementation and Modeling. The system configuration and hardware consumption of TT-GNN are shown in Table 5. Power and area statistics of customized modules are obtained from synthesizing RTL implementation using Synopsys Design Compiler under TSMC 22nm standard cell library. The latency, power, as well as area of SRAM modules, are simulated with CACTI [28]. For performance and energy-efficiency evaluation, we implement a custom simulator that is integrated with the software framework to capture real training traces.

Table 5: Configurations, Power, and Area of TT-GNN under 22nm Technology and 1GHz Frequency.

Hardware Module	Configuration	Power(mW)	Area(mm ²)
Contraction Unit	16×16 FP16 MAC	441.94	0.41
PE Array	32×16 FP16 MAC	968.97	0.93
SFU	16 Exp, 16 Div	78.21	0.071
SRAM Buffer	38.125MB	1048.44	27.15

Hardware Baseline We first compare TT-GNN with a standard CPU-GPU training system. The four smaller benchmarks are evaluated on a single Nvidia 3090 GPU and an AMD Ryzen Threadripper 3970X 32-Core CPU, while the largest ogbn-papers100M is evaluated on a A100 GPU. In the baseline system, graphs are originally stored in host DRAM and loaded to device memory during training. Sub-graph sampling is offloaded to CPU, and we issue multiple threads to achieve the shortest sampling latency. We also include a

A100 GPU implementation of TT-GNN as another baseline to illustrate the advantage of using the proposed accelerator architecture and customized dataflow. For TT-GNN, the TT-format embedding can be stored on-chip, while the graph edge list and sub-graph sampling are executed on the host system. For performance comparison, we scale up TT-GNN's configuration to have the same peak computation throughput as the 3090 GPU.

8 EVALUATION RESULTS

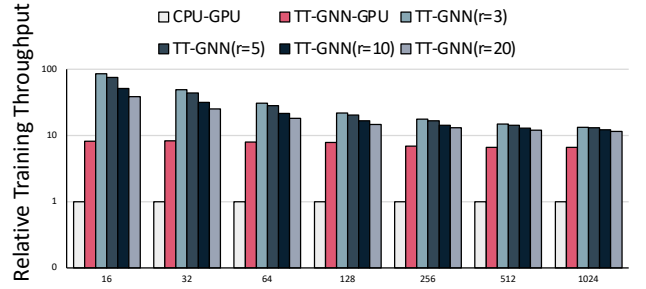


Figure 12: Relative training throughput comparison on ogbn-papers100M.

8.1 Performance Evaluation

8.1.1 Training Throughput. We first compare the training performance between TT-GNN and the baseline CPU-GPU training system across different benchmarks and minibatch sizes. As shown in Figure 11 and 12, overall, TT-GNN-GPU is 1.21 ~ 8.26× faster than the CPU-GPU training baseline, and TT-GNN accelerator achieves 1.55 ~ 4210× throughput improvement.

For TT-GNN-GPU, the speedup comes from reducing CPU to GPU data transfer as the TT-format embedding can be stored in GPU HBM. However, due to the overhead of TT computation, the improvement is compromised, especially on smaller batch sizes. TT-GNN accelerator significantly improves the because of the following advantages. First, TT-GNN avoids fetching off-chip embedding through effective compression and the proposed Hybrid Minibatch-Microbatch dataflow. This is a game-changing difference because even for TT-GNN-GPU, we have to constantly access off-chip HBM

between every two kernels or even within a single kernel to manage data. On the contrary, TT-GNN-GPU can be considered as a complete fusion of all kernels within a microbatch. Furthermore, TT (de)compression has limited performance on GPU due to its limited problem size and challenging intermediate result reuse pattern. In TT-GNN accelerator, we address this with dedicated architecture and dataflow as proposed in Section 5.3. Finally, we leverage aggregation redundancy within each microbatch subgraph by caching the partial sums during neighbor aggregation and gradient scattering. Besides the overall trend, we also observe that TT-GNN achieves higher speedup under smaller batchsizes. This is because GPU suffers from severe resource under-utilization when the batchsize is small. The fixed latency such as kernel launching overhead and idleness caused by subgraph sampling also accounts for a larger fraction with small batchsizes.

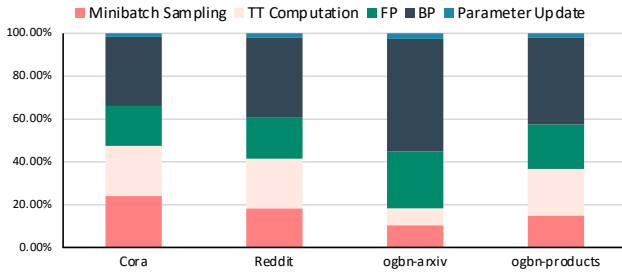


Figure 13: Training latency breakdown of one minibatch.

8.1.2 Latency Breakdown. Figure 13 presents the average latency breakdown of executing one minibatch. Overall, the minibatch sampling and TT computation have a comparable latency with forward and backward propagation. This supports our pipelined design to fully hide the subgraph preparation overhead. Besides, on benchmarks such as ogbn-arxiv, the number of input nodes per destination node is much less. As a result, the computation is more dominated by FC layers, leading to a larger portion of forward and backward propagation. Note that, the TT-rank value will significantly change the complexity of Tensor-train contraction, and thus affecting the latency of TT decompression and TT-gradient computation. In TT-GNN, we reduce this impact by caching the prefix contraction result during the forward propagation, and reuse it for TT-gradient computation. Overall, as we discussed in Section 5.3, we are able to generate all the required Tensor-trains with a complexity equal to contracting only two tensor-trains.

8.1.3 Impact of Different Techniques. We use the Reddit dataset to demonstrate the effect of the proposed techniques. The other benchmarks exhibit similar breakdown fractions but have different absolute values. As shown in Figure 14, we break down the speedup of TT-GNN to the specific techniques we discussed above. Overall, the specialized accelerator design and on-chip learning mechanism bring 11.2× of performance improvements. With limited SRAM buffer capacity, the on-chip learning is only possible within the microbatches. This is the core value of the proposed Hybrid Minibatch-Microbatch dataflow. The performance improvement is further amplified by 1.25× with the microbatch scheduling strategy, and by 1.11× with the aggregation partial sum reuse. In our

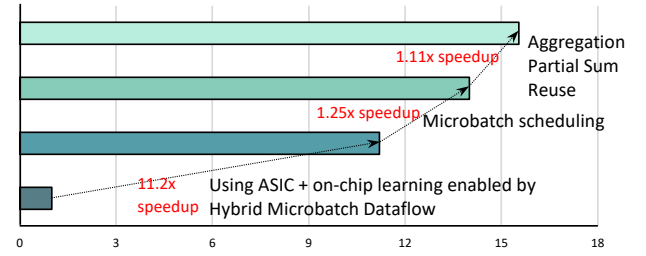


Figure 14: Speedup breakdown of TT-GNN on Reddit.

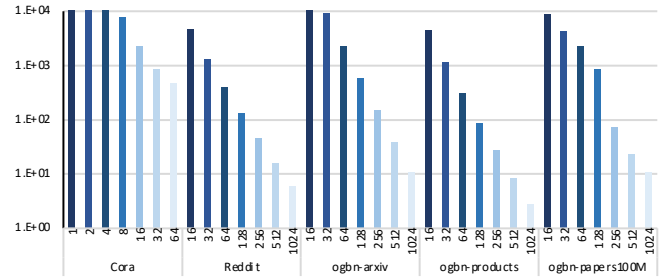


Figure 15: Relative energy-efficiency improvements of TT-GNN over baseline CPU-GPU training system.

experiment, we observe that the benefit of reusing intermediate partial sum is less than the reported number in literature [2, 13]. This is because we can only operate on the microbatch-level compute graph, where neighbor sharing is less effective.

8.2 Energy-efficiency

Finally, we show the energy-efficiency improvements of TT-GNN with Figure 15. As we can see, TT-GNN has 2.83× to orders of magnitude better energy efficiency than the baseline system. Apart from the natural benefit of using specialized dataflow and ASIC design, the most important advantage is that we completely avoid off-chip memory access during the microbatch execution. This is a significant portion of the energy consumption in the original training setting. Similar to the speedup analysis, the advantage of a dedicated on-chip training accelerator over GPU is larger on smaller batchsizes, as GPU suffers from resource under-utilization and fixed energy consumption.

9 RELATED WORK

Tensorized Neural Network Tensor-train Decomposition has been widely applied in Neural Networks including CNNs [30], RNNs [44], Recommendation Models [46], and Transformers [17]. The unique computation pattern of Tensor-train has inspired research efforts on customized accelerator design [6] for these Tensorized Neural Networks (TNNs).

GNN Training Accelerator There exists a series of works aimed at scaling Graph Neural Network training. To start with, HyScale-GNN [21] introduces a single-node heterogeneous architecture that utilizes both processors and accelerators to train a large-scale GNN. Another line of work moves computations closer to memory and storage such that the graph learning are handled directly at the place where the graph is stored. The motivation of these works

is to mitigate I/O bottleneck caused by storing large graphs. For example, SmartSAGE [19] implements the subgraph sampling operation inside SSDs, and GLIST [20] designs a customized graph learning accelerator implemented in the storage. Ginex [34] further optimizes the SSD-based GNN training pipeline and optimizes the cache mechanism. Similarly, GNNear [50] handles full batch GNN training with a heterogeneous DIMM-based architecture and acceleration engine. From the graph sampling perspective, TT-GNN is orthogonal to these approaches. For example, a system can incorporate in-storage subgraph sampling like SmartSAGE while leveraging the compressed TT-format embedding and using TT-GNN accelerator to further improve training performance. From the graph training perspective, TT-GNN directly addresses the I/O bottleneck at the root cause. If the graph embedding can be directly stored inside the on-chip buffer, then there is no need to enable graph learning inside large memories, which also avoids changing the storage architecture as well as the system software stack.

Another series of work aims at improving GNN training efficiency on existing single and multi-GPU systems [22, 24, 25, 35, 37, 40, 42, 43], with different focuses including sampling algorithm, workload partitioning strategy, caching mechanism, data and computational parallelism, and so on. While these works provide solid improvements, none of them addresses the explosion issue of graph embedding, which requires changing the embedding representation in the first place. Adopting a compressed format node embedding brings practical but limited improvements as we show in section 8. Therefore, with TT-GNN we further show that a careful software and hardware co-design is necessary in order to fully exploit the algorithmic benefit.

10 CONCLUSION

In this paper, we propose TT-GNN, a training system that adopts Tensor-train Decomposition to compress the memory-consuming feature embedding matrix, which leads to an on-chip learning implementation. TT-GNN adaptively breaks down a minibatch into smaller microbatches that can be fitted on-chip. The microbatch composition and scheduling order are designed to maximize data reuse and reduce redundant computations both across and within microbatches. We also propose a unified algorithm to jointly handle TT decompression during forward propagation and TT gradient derivation during backward propagation. Combining the software and hardware optimizations, the proposed software-hardware solution is able to outperform existing CPU-GPU training systems on both training performance (1.55~4210 $\times$) and energy efficiency (2.83~2254 $\times$).

REFERENCES

- [1] Youhui Bai, Cheng Li, Zhiqi Lin, Yufei Wu, Youshan Miao, Yunxin Liu, and Yinlong Xu. 2021. Efficient Data Loader for Fast Sampling-Based GNN Training on Large Graphs. *IEEE Transactions on Parallel and Distributed Systems* 32, 10 (2021), 2541–2556. <https://doi.org/10.1109/TPDS.2021.3065737>
- [2] C. Chen, K. Li, Y. Li, and X. Zou. 2022. ReGNN: A Redundancy-Eliminated Graph Neural Networks Accelerator. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 429–443. <https://doi.org/10.1109/HPCA53966.2022.00039>
- [3] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. *CoRR* abs/1905.07953 (2019). arXiv:1905.07953 <http://arxiv.org/abs/1905.07953>
- [4] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. *CoRR* abs/1905.07953 (2019). arXiv:1905.07953 <http://arxiv.org/abs/1905.07953>
- [5] Matthieu Courbariaux and Yoshua Bengio. 2016. BinaryNet: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1. *CoRR* abs/1602.02830 (2016). arXiv:1602.02830 <http://arxiv.org/abs/1602.02830>
- [6] Chunhua Deng, Fangxuan Sun, Xuehai Qian, Jun Lin, Zhongfeng Wang, and Bo Yuan. 2019. TIE: Energy-Efficient Tensor Train-Based Inference Engine for Deep Neural Network. In *Proceedings of the 46th International Symposium on Computer Architecture (Phoenix, Arizona) (ISCA '19)*. Association for Computing Machinery, New York, NY, USA, 264–278. <https://doi.org/10.1145/3307650.3322258>
- [7] David Duvenaud, Dougal Maclaurin, Jorge Aguilera-Iparraguirre, Rafael Gómez-Bombarelli, Timothy Hirzel, Alán Aspuru-Guzik, and Ryan P. Adams. 2015. Convolutional Networks on Graphs for Learning Molecular Fingerprints. *CoRR* abs/1509.09292 (2015). arXiv:1509.09292 <http://arxiv.org/abs/1509.09292>
- [8] Mateusz Gabor and Rafal Zdunek. 2022. Convolutional Neural Network Compression via Tensor-Train Decomposition on Permuted Weight Tensor with Automatic Rank Determination. In *Computational Science – ICCS 2022*, Derek Groen, Clélia de Mulatier, Maciej Paszynski, Valeria V. Krzhizhanovskaya, Jack J. Dongarra, and Peter M. A. Sloot (Eds.). Springer International Publishing, Cham, 654–667.
- [9] Kai Guo and Markus J. Buehler. 2022. Rapid prediction of protein natural frequencies using graph neural networks. *Digital Discovery* 1 (2022), 277–285. Issue 3. <https://doi.org/10.1039/D1DD00007A>
- [10] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. *CoRR* abs/1706.02216 (2017). arXiv:1706.02216 <http://arxiv.org/abs/1706.02216>
- [11] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. *arXiv preprint arXiv:2005.00687* (2020).
- [12] Qian Huang, Horace He, Abhay Singh, Ser-Nam Lim, and Austin Benson. 2021. Combining Label Propagation and Simple Models out-performs Graph Neural Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=8E1-f3VhX1o>
- [13] Zhihao Jia, Sina Lin, Rex Ying, Jiaxuan You, Jure Leskovec, and Alex Aiken. 2019. Redundancy-Free Computation Graphs for Graph Neural Networks. arXiv:1906.03707 [cs.LG]
- [14] Tim Kaler, Nickolas Stathas, Anne Ouyang, Alexandros-Stavros Iliopoulos, Tao B. Schardl, Charles E. Leiserson, and Jie Chen. 2021. Accelerating Training and Inference of Graph Neural Networks with Fast Sampling and Pipelining. *CoRR* abs/2110.08450 (2021). arXiv:2110.08450 <https://arxiv.org/abs/2110.08450>
- [15] George Karypis and Vipin Kumar. 1998. A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs. *SIAM J. Sci. Comput.* 20, 1 (Dec. 1998), 359–392.
- [16] Bruce Khailany, Haoxing Ren, Steve Dai, Saad Godil, Ben Keller, Robert Kirby, Alicia Klinefelter, Rangharajan Venkatesan, Yanqing Zhang, Bryan Catanzaro, and William J. Dally. 2020. Accelerating Chip Design With Machine Learning. *IEEE Micro* 40, 6 (2020), 23–32. <https://doi.org/10.1109/MM.2020.3026231>
- [17] Valentin Khrulkov, Oleksii Hrinchuk, L. Mirvakhabova, and I. Oseledets. 2019. Tensorized Embedding Layers for Efficient Model Compression. *ArXiv* abs/1901.10787 (2019).
- [18] Diederik P. Kingma and Jimmy Ba. 2017. Adam: A Method for Stochastic Optimization. arXiv:1412.6980 [cs.LG]
- [19] Yunjae Lee, Jinha Chung, and Minsoo Rhu. 2022. SmartSAGE: Training Large-Scale Graph Neural Networks Using in-Storage Processing Architectures. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (New York, New York) (ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 932–945. <https://doi.org/10.1145/3470496.3527391>
- [20] Cangyuan Li, Ying Wang, Cheng Liu, Shengwen Liang, Huawei Li, and Xiaowei Li. 2021. GLIST: Towards In-Storage Graph Learning. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 225–238. <https://www.usenix.org/conference/atc21/presentation/li-cangyuan>
- [21] Yi-Chien Lin and Viktor Prasanna. 2023. HyScale-GNN: A Scalable Hybrid GNN Training System on Single-Node Heterogeneous Architecture. arXiv:2303.00158 [cs.DC]
- [22] Zhiqi Lin, Cheng Li, Youshan Miao, Yunxin Liu, and Yinlong Xu. 2020. PaGraph: Scaling GNN Training on Large Graphs via Computation-Aware Caching. In *Proceedings of the 11th ACM Symposium on Cloud Computing (Virtual Event, USA) (SoCC '20)*. Association for Computing Machinery, New York, NY, USA, 401–415. <https://doi.org/10.1145/3419111.3421281>
- [23] Tianfeng Liu, Yangrui Chen, Dan Li, Chuan Wu, Yibo Zhu, Jun He, Yanghua Peng, Hongzheng Chen, Hongzhi Chen, and Chuanxiong Guo. 2021. BGL: GPU-Efficient GNN Training by Optimizing Graph Data I/O and Preprocessing. *CoRR* abs/2112.08541 (2021). arXiv:2112.08541 <https://arxiv.org/abs/2112.08541>
- [24] Tianfeng Liu, Yangrui Chen, Dan Li, Chuan Wu, Yibo Zhu, Jun He, Yanghua Peng, Hongzheng Chen, Hongzhi Chen, and Chuanxiong Guo. 2021. BGL:

- GPU-Efficient GNN Training by Optimizing Graph Data I/O and Preprocessing. arXiv:2112.08541 [cs.LG]
- [25] Xin Liu, Mingyu Yan, Shuhan Song, Zhengyang Lv, Wenming Li, Guangyu Sun, Xiaochun Ye, and Dongrui Fan. 2022. GNNsampler: Bridging the Gap between Sampling Algorithms of GNN and Hardware. arXiv:2108.11571 [cs.LG]
 - [26] Daniela Sánchez Lopera, Lorenzo Servadei, Gamze Naz Kiprit, Souvik Hazra, Robert Wille, and Wolfgang Ecker. 2021. A Survey of Graph Neural Networks for Electronic Design Automation. In *2021 ACM/IEEE 3rd Workshop on Machine Learning for CAD (MLCAD)*. 1–6. <https://doi.org/10.1109/MLCAD52597.2021.9531070>
 - [27] Yuzhe Ma, Zhuolun He, Wei Li, Lu Zhang, and Bei Yu. 2020. Understanding Graphs in EDA: From Shallow to Deep Learning. *Proceedings of the 2020 International Symposium on Physical Design* (2020).
 - [28] Naveen Muralimanohar, Rajeev Balasubramanian, and Norman Jouppi. 2009. Cacti 6.0: A tool to model large caches. *HP Laboratories* (01 2009).
 - [29] Alexander Novikov, Dmitry Podoprikin, Anton Osokin, and Dmitry P. Vetrov. 2015. Tensorizing Neural Networks. *CoRR* abs/1509.06569 (2015). arXiv:1509.06569 <http://arxiv.org/abs/1509.06569>
 - [30] Alexander Novikov, Dmitry Podoprikin, Anton Osokin, and Dmitry P. Vetrov. 2015. Tensorizing Neural Networks. *CoRR* abs/1509.06569 (2015). arXiv:1509.06569 <http://arxiv.org/abs/1509.06569>
 - [31] Charles C. Onu, Jacob E. Miller, and Doina Precup. 2020. A Fully Tensorized Recurrent Neural Network. *CoRR* abs/2010.04196 (2020). arXiv:2010.04196 <https://arxiv.org/abs/2010.04196>
 - [32] I. V. Oseledets. 2011. Tensor-Train Decomposition. *SIAM Journal on Scientific Computing* 33, 5 (2011), 2295–2317. <https://doi.org/10.1137/090752286> arXiv:https://doi.org/10.1137/090752286
 - [33] Mike O'Connor, Niladrish Chatterjee, Donghyuk Lee, John Wilson, Aditya Agrawal, Stephen W. Keckler, and William J. Dally. 2017. Fine-Grained DRAM: Energy-Efficient DRAM for Extreme Bandwidth Systems. In *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 41–54.
 - [34] Yeonhong Park, Sunhong Min, and Jae W. Lee. 2022. Ginex: SSD-Enabled Billion-Scale Graph Neural Network Training on a Single Machine via Provably Optimal in-Memory Caching. *Proc. VLDB Endow.* 15, 11 (jul 2022), 2626–2639. <https://doi.org/10.14778/3551793.3551819>
 - [35] Sandeep Polisetty, Juelin Liu, Kobi Falus, Yi Ren Fung, Seung-Hwan Lim, Hui Guan, and Marco Serafini. 2023. GSplit: Scaling Graph Neural Network Training on Large Graphs via Split-Parallelism. arXiv:2303.13775 [cs.DC]
 - [36] Manon Réau, Nicolas Renaud, Li C Xue, and Alexandre M J J Bonvin. 2022. DeepRank-GNN: a graph neural network framework to learn patterns in protein–protein interfaces. *Bioinformatics* 39, 1 (11 2022). <https://doi.org/10.1093/bioinformatics/btac759> arXiv:https://academic.oup.com/bioinformatics/article-pdf/39/1/btac759/48448995/btac759_supplementary_data.pdf btac759.
 - [37] Shihui Song and Peng Jiang. 2022. Rethinking Graph Data Placement for Graph Neural Network Training on Multiple GPUs. In *Proceedings of the 36th ACM International Conference on Supercomputing (Virtual Event) (ICS '22)*. Association for Computing Machinery, New York, NY, USA, Article 39, 10 pages. <https://doi.org/10.1145/3524059.3532384>
 - [38] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. arXiv:1710.10903 [stat.ML]
 - [39] Minjie Wang, Lingfan Yu, Da Zheng, Quan Gan, Yu Gai, Zihao Ye, Mufei Li, Jinjing Zhou, Qi Huang, Chao Ma, Ziyue Huang, Qipeng Guo, Hao Zhang, Haibin Lin, Junbo Zhao, Jinyang Li, Alexander J. Smola, and Zheng Zhang. 2019. Deep Graph Library: Towards Efficient and Scalable Deep Learning on Graphs. *CoRR* abs/1909.01315 (2019). arXiv:1909.01315 <http://arxiv.org/abs/1909.01315>
 - [40] Qiang Wang, Yanfeng Zhang, Hao Wang, Chaoyi Chen, Xiaodong Zhang, and Ge Yu. 2022. NeutronStar: Distributed GNN Training with Hybrid Dependency Management. In *Proceedings of the 2022 International Conference on Management of Data (Philadelphia, PA, USA) (SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 1301–1315. <https://doi.org/10.1145/3514221.3526134>
 - [41] Shihui Wu, Wentao Zhang, Fei Sun, and Bin Cui. 2020. Graph Neural Networks in Recommender Systems: A Survey. arXiv:2011.02260 [cs.IR]
 - [42] Jianbang Yang, Dahai Tang, Xiaoni Song, Lei Wang, Qiang Yin, Rong Chen, Wenyan Yu, and Jingren Zhou. 2022. GNNLab: A Factored System for Sample-Based GNN Training over GPUs. In *Proceedings of the Seventeenth European Conference on Computer Systems (Rennes, France) (EuroSys '22)*. Association for Computing Machinery, New York, NY, USA, 417–434. <https://doi.org/10.1145/3492321.3519557>
 - [43] Shuangyan Yang, Minjia Zhang, Wenqian Dong, and Dong Li. 2023. Betty: Enabling Large-Scale GNN Training with Batch-Level Graph Partitioning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (Vancouver, BC, Canada) (ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 103–117. <https://doi.org/10.1145/3575693.3575725>
 - [44] Yinchong Yang, Denis Krompass, and Volker Tresp. 2017. Tensor-Train Recurrent Neural Networks for Video Classification. *CoRR* abs/1707.01786 (2017). arXiv:1707.01786 <http://arxiv.org/abs/1707.01786>
 - [45] Ziyue Yang, Maghesree Chakraborty, and Andrew D White. 2020. Predicting Chemical Shifts with Graph Neural Networks. *bioRxiv* (2020). <https://doi.org/10.1101/2020.08.26.267971> arXiv:https://www.biorxiv.org/content/early/2020/08/27/2020.08.26.267971.full.pdf
 - [46] Chunxing Yin, Bilge Acun, Xing Liu, and Carole-Jean Wu. 2021. TT-Rec: Tensor Train Compression for Deep Learning Recommendation Models. arXiv:2101.11714 [cs.LG]
 - [47] Chunxing Yin, Da Zheng, Israt Nisa, Christos Faloutsos, George Karypis, and Richard Vuduc. 2022. Nimble GNN Embedding with Tensor-Train Decomposition. arXiv:2206.10581 [cs.LG]
 - [48] Lizhi Zhang, Zhiqian Lai, Shengwei Li, Yu Tang, Feng Liu, and Dongsheng Li. 2021. 2PGraph: Accelerating GNN Training over Large Graphs on GPU Clusters. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)*. 103–113. <https://doi.org/10.1109/Cluster48925.2021.00036>
 - [49] Hang Zhao, Yujing Wang, Juanyong Duan, Congrui Huang, Defu Cao, Yunhai Tong, Bixiong Xu, Jing Bai, Jie Tong, and Qi Zhang. 2020. Multivariate Time-series Anomaly Detection via Graph Attention Network. arXiv:2009.02040 [cs.LG]
 - [50] Zhe Zhou, Cong Li, Xuechao Wei, Xiaoyang Wang, and Guangyu Sun. 2022. GNNear: Accelerating Full-Batch Training of Graph Neural Networks with Near-Memory Processing. arXiv:2111.00680 [cs.LG]